Operating Systems Case Study

Race Conditions and Synchronization in xv6

A practical study of the physical-page allocator

Submitted by

Name	Roll number
L Nithish Kumar Goud	160124733327
G Ishan	160124733322
Santhosh	160124733332
Venkat	160124733340

Department of Computer Science and Engineering

Institution: _____

Faculty: _____

Semester / Section: _____

Academic year: _____

Experiment evidence: 24 October 2026
Report prepared: 25 October 2026

Certificate and approval details

Insert the department-approved certificate wording in the space below before submission. The supplied guidelines require a certificate but do not provide its wording. This page supplies the identification and approval fields without asserting that certification has been granted.

Case study identification

Field	Details
Title	Race Conditions and Synchronization in xv6
Course	Operating Systems
Institution	_____
Department	Computer Science and Engineering
Faculty / designation	_____
Semester / section	_____
Academic year	_____

Department-approved certificate wording

_____

_____

_____

Approval fields

Faculty signature	Department approval	Date
_____	_____	_____

Members and contributions

The members below are supplied by the team. Individual technical contributions have not yet been confirmed. Complete the contribution fields using actual work performed, reviewed, or presented by each person; preparation responsibilities alone do not establish completed contributions.

Table 1. Team members and contribution confirmation

No.	Name / roll number	Actual contribution
1	L Nithish Kumar Goud 160124733327	To be confirmed by member
2	G Ishan 160124733322	To be confirmed by member
3	Santhosh 160124733332	To be confirmed by member
4	Venkat 160124733340	To be confirmed by member

Implementation ownership and evidence

The supplied project includes an xv6 C workload, a host-side C runner, recorded QEMU logs, source diffs, and restoration checks. The report describes those artifacts and their observed results. It does not assign their authorship to a particular team member without confirmation.

Standard xv6 kernel mechanisms are upstream code. The study exercises the allocator with race_test.c and temporarily removes four existing allocator lock calls before restoring them. The host helper automates execution and evidence collection; it is not itself the synchronization mechanism.

Before submission

Each member should check their identifying details, confirm their contribution entry, understand the practical procedure, and be able to explain why the free-list updates need protection. Faculty and signature fields require completion by the appropriate people.

Index

Contents

Section	Page
Problem statement	5
Introduction and objectives	6
Problems caused by race conditions	7
Existing xv6 system and background	8
Synchronization types: exclusion	9
Synchronization types: coordination	10
Proposed solution and working flow	11
Practical environment and protocol	12
Shared data and protected code	13
C workload and checking logic	14
Execution paths and reproduction	15
Possible allocator interleavings	16
Original-lock baseline	17
Deliberate lock removal	18
Unlocked-run evidence	19
Restoration and verification	20
Results and interpretation	21
All attempts: A01-B05	22
All attempts: B06-C10	23
Conclusion and future scope	24
References	25
Appendix A: complete C workload	26-28

1. Problem statement

An operating system allows several activities to progress concurrently. On a multicore machine, two CPUs can execute kernel code at the same time. A shared data structure can become inconsistent when one CPU reads or changes it while another CPU is partway through an update. The resulting behavior can depend on timing rather than on the intended program logic.

This study asks how synchronization protects the xv6 physical-page allocator when concurrent processes repeatedly request and release memory. The shared structure is kmem.freelist, a linked list of available physical pages. The investigation compares the normal allocator, a deliberately unlocked experimental variant, and the allocator after its original locking is restored.

Practical question

Under the same four-CPU workload, do the recorded outcomes change when the existing allocator lock calls are removed and then restored? The study uses a healthy original-lock baseline as a control, changes only the intended allocator calls for the unlocked phase, and checks restoration before the final phase.

What is being tested

The workload runs as four xv6 user processes, but the deliberately unprotected shared data is in the kernel. The children have separate virtual address spaces. Their memory-management system calls converge on the same physical allocator, making kernel-level coordination necessary even though each child expects a private buffer.

Scope and success criteria

Success means explaining the shared-data invariant, identifying the protected operations, documenting the controlled change, preserving every attempt, and connecting observations to the source without inventing an internal execution trace. Other synchronization methods are explained for comparison; only the existing xv6 allocator spinlock is evaluated experimentally.

The measurements describe a specific revision, workload, and emulator configuration. They provide evidence for these tested conditions rather than proof that every locked program is correct or that every unlocked execution must fail.

2. Introduction, objectives and motivation

A race condition occurs when correctness depends on the relative timing of concurrent activities. A data race is a more specific case involving conflicting memory accesses that lack the required synchronization. At least one conflicting access is a write. A race condition can also involve a longer sequence of individually protected operations when that sequence is not coordinated as a whole.

A shared resource is data or a device that multiple activities can access. A critical section is the part of a program that accesses shared state whose invariant must be preserved. Mutual exclusion permits only one participant at a time in the sections guarded by the same lock. Atomicity means an operation is indivisible with respect to the other participating operations; a source statement is not automatically atomic.

Table 2. Illustrative lost update: counter starts at 0

Step	Activity A	Activity B
1	Read counter as 0	
2		Read counter as 0
3	Compute 0 + 1	Compute 0 + 1
4	Store 1	
5		Store 1

Two increments should produce 2, but this possible schedule leaves 1. The expression counter++ usually involves a read, a calculation, and a write. This table illustrates the idea; it is not a safe executable C experiment with an ordinary unsynchronized shared variable, because C data races have undefined behavior.

Objectives and motivation

The objectives are to recognize shared kernel state, explain harmful interleavings, understand the locking mechanism, and compare recorded outcomes before removal, after removal, and after restoration. The motivation is to connect synchronization theory to a small, inspectable operating system and a repeatable C workload.

3. Problems caused by race conditions

A race may produce the right output in many executions and fail only under a particular schedule. This makes debugging difficult: additional logging, a different compiler, or a different CPU count can change timing. A successful run therefore cannot establish that an unsynchronized algorithm is safe.

Table 3. Consequences of unsafe concurrent access

Problem	Explanation and example
Lost update	One write overwrites another completed update, such as a missing counter increment or a lost free-list insertion.
Inconsistent state	A reader observes only part of a multi-step update and uses a structure whose invariant is temporarily broken.
Corruption	An unintended writer changes data that another process expects to own; the workload checks for changed byte tags.
Resource errors	A possible allocator race can lose a free page or return a page incorrectly. The workload does not identify every such error.
Crash or trap	A damaged structure may lead to invalid memory access. A panic alone does not identify the precise race.
Unpredictable behavior	The timing of conflicting accesses changes the observed outcome between executions.

Race condition versus deadlock

A race concerns incorrect interaction between concurrent operations. A deadlock concerns participants waiting indefinitely for conditions that cannot be satisfied, such as two activities holding different locks while each waits for the other. Locks can prevent a race but can introduce deadlock if used incorrectly. A host timeout is an observation of missing completion within a limit; it is not proof of deadlock.

Misleading fixes

Adding a delay or printing more output may change timing without fixing the shared-data rule. volatile does not provide mutual exclusion or the required inter-thread ordering. Disabling interrupts on one CPU does not stop another CPU from changing the same data. A correct solution must coordinate every relevant access to the invariant.

4. Existing xv6 system and background

xv6 is a small Unix-like teaching operating system. The fixed RISC-V source revision used here already contains synchronization. Its physical allocator manages 4096-byte pages through a global linked list. Each free page begins with a struct run containing the next pointer, and kmem.freelist identifies the first free page.

kalloc() removes the head page. kfree() inserts a page at the head after validating and filling it with a diagnostic pattern. Both operations acquire kmem.lock while changing the shared list. The practical problem is deliberately introduced by removing those calls; standard xv6 is not being described as an allocator without locks.

Table 4. Existing mechanisms and their responsibilities

Mechanism	Purpose in the examined source
kmem.lock	Serializes allocator free-list updates in kalloc() and kfree().
Atomic exchange	Makes ownership acquisition an indivisible read-modify-write operation.
Acquire/release ordering	Constrains memory ordering around the protected operations.
push_off() / pop_off()	Manage nested local interrupt disabling while spinlocks are held.
Sleep-lock	Allows a waiting process to sleep for suitable longer operations; distinct from the allocator spinlock.

Background search and existing-system drawbacks

The local source is the authority for exact implementation details. The MIT xv6 locking chapter provides background on locks, invariants, and contention. POSIX synchronization documentation supplies comparison terminology for user-space primitives. The general methods are not all implemented by this case study.

A single allocator lock is easy to reason about, but simultaneous requests contend for it and enter the protected list updates one at a time. Busy waiting can consume CPU cycles. These are design tradeoffs rather than measured performance findings here. Removing the lock sacrifices correctness instead of solving contention. Designs such as per-CPU free lists require a separate correctness argument and are future work.

5. Synchronization types: mutual exclusion

Synchronization coordinates access to shared state or the order of events. Mutual exclusion and condition synchronization solve related but different problems. The correct primitive depends on the resource, expected waiting time, execution context, and whether the protected operation may sleep.

Table 5. Methods for controlling access

Method	Typical use	Important limitation
Mutex	One owner protects a shared object; contended blocking mutexes can put threads to sleep.	All participants must use the same discipline; ownership and lock order matter.
Spinlock	A CPU repeatedly tries an atomic operation; useful for short kernel sections where sleeping is unsuitable.	Waiting consumes CPU time; do not sleep while holding an ordinary xv6 spinlock.
Reader-writer lock	Several readers may enter together; a writer needs exclusive access.	Not a free speed improvement; fairness and starvation depend on policy.
Atomic operation	An indivisible load, store, exchange, or read-modify-write on a supported object.	An atomic field alone does not preserve a multi-field invariant.
Binary semaphore	A permit count of zero or one can gate access.	Ownership semantics differ from a mutex; signaling must match the protocol.

Why the allocator uses a spinlock

The free-list push and pop are short operations in a kernel that can run on multiple CPUs. The lock protects the relationship between the head and the next pointer while an update is in progress. The spinlock is an existing upstream mechanism retained or restored by this study, not a new implementation of all synchronization types.

Atomicity is a building block

A plain check followed by a plain assignment is insufficient to claim lock ownership: two CPUs could both observe the lock as free. Atomic exchange combines the check and change. The critical section still needs the correct data-structure algorithm and the correct memory-ordering rules.

6. Synchronization types: event coordination

Table 6. Methods for coordinating progress

Method	Purpose	Correct-use rule
Counting semaphore	Tracks available permits, such as a bounded resource pool; waiting consumes a permit and posting returns one.	Choose the initial count and balance operations; it does not automatically protect every field.
Condition variable	Waits for a predicate, such as a nonempty queue, together with a mutex.	Check the predicate in a while loop; waiting releases and later reacquires the mutex.
Monitor	Groups shared state with operations under an exclusion rule, often using condition variables.	Maintain its invariant and use the language or library wait protocol correctly.
Barrier	Stops participants at a phase boundary until the required participants arrive.	It does not provide mutual exclusion for updates performed inside a phase.
Sleep-lock	Lets a waiting xv6 process sleep while another process owns the resource.	Suitable contexts may sleep; use the xv6 sleep/wakeup rules and avoid incompatible lock nesting.

Mutual exclusion and ordering are complementary

A mutex answers who may modify an object now. A condition variable answers when a participant may proceed after a state change. A barrier answers whether all participants have reached a phase boundary. These primitives are not interchangeable. For example, a barrier before two increments can make their overlap more likely without making the increments safe.

A producer-consumer queue commonly uses a mutex for the queue invariant and conditions for empty or full states. The condition wait must atomically release the mutex and begin waiting, then reacquire the mutex before returning. The predicate is checked again because a wakeup does not guarantee it is still true.

These examples explain selection principles. This experiment does not compare their speed or execute separate mutex, semaphore, barrier, or reader-writer implementations. The practical results concern the existing xv6 allocator spinlock only.

7. Proposed solution and working flow

The solution is to retain or restore kmem.lock around every relevant free-list update. With consistent use, one CPU completes a push or pop before another CPU begins a conflicting update. The lock protects the list invariant, not merely the variable name. Correctness also assumes the code inside the critical section is correct and that callers respect allocation ownership.

```
acquire(&kmem.lock)
    read or modify kmem.freelist and list links
release(&kmem.lock)
```

Figure 1. The allocator critical section, expressed as a source-level flow.

How acquisition and release work

In the examined source, acquire() first invokes push_off() and checks whether the current CPU already holds the lock. It repeatedly calls __atomic_exchange_n with __ATOMIC_ACQUIRE until the old lock value is zero. It then records the owner CPU. Atomic exchange prevents two CPUs from winning the same ownership transition.

release() checks ownership, clears the owner record, stores zero with __ATOMIC_RELEASE, and invokes pop_off(). Acquire and release ordering keep the protected memory accesses on the appropriate side of the ownership transition. The hardware and compiler must respect these constraints; mutual exclusion cannot be built by treating ordinary loads and stores as an atomic pair.

Interrupt handling and correct lock use

Nested interrupt handling records the prior interrupt state and disables local interrupts while needed. This helps avoid same-CPU interruption and re-entry problems while holding a spinlock. It does not replace cross-CPU locking. Removing allocator acquire/release calls also removes their interrupt-handling effects where no outer lock is active, so the experiment does not isolate those effects independently.

Use a consistent lock order, keep the section short, and match each acquisition with release on every applicable path. Avoid sleeping while holding an ordinary xv6 spinlock. The working experimental flow is original locking, controlled removal, identical workload, exact restoration, and repeated measurement.

8. Practical environment and protocol

Table 7. Fixed experimental settings

Setting	Recorded value
Host environment	Ubuntu 24.04 under WSL2 on Windows
Source revision	f780d7850765d74cbdcd1fff4b822d8829f7c9bd
Guest / emulator	RISC-V xv6; QEMU 8.2.2
Toolchain	RISC-V GCC 13.3.0
Guest resources	4 virtual CPUs; 128 MiB RAM
Workload	4 children; 8 pages per child; 200 rounds
Page / allocation size	4096 bytes; 32768 bytes per child per round
Attempt protocol	Fresh QEMU guest; one stress invocation per attempt
Limits	30 seconds to shell; 60 seconds after stress command
Measured run set	member1_completed/c_validation; 10 attempts per phase

The workload and kernel modifications are C. The host helper, member1_tools.c, builds and runs the guest, collects console output, enforces time limits, and packages evidence. It runs outside xv6. The helper is not the locking solution; the allocator spinlock is the mechanism under study.

The three phases use the same workload and guest configuration. A01-A10 use original locks, B01-B10 use the deliberately unlocked allocator, and C01-C10 follow restoration. Sanity and smoke checks are separate from these 30 measured attempts. There was no kernel timing instrumentation.

The saved console logs and CSV are the primary evidence. The run set is historical evidence from 24 October 2026; report preparation does not constitute a new execution. Earlier experiments exist in a different folder and are not merged into these results.

9. Practical shared data and protected code

```
struct run {
  struct run *next;
};

struct {
  struct spinlock lock;
  struct run *freelist;
} kmem;
```

Figure 2. Shared allocator representation from the examined kalloc.c.

kmem.freelist points to the first available physical page. For each free page, next identifies the next free page. The invariant requires a coherent chain of available pages: allocation removes one page, and freeing adds one valid page. The normal list updates are protected by the same lock in both functions.

```
/* kalloc(): protected pop */
acquire(&kmem.lock);
r = kmem.freelist;
if (r)
  kmem.freelist = r->next;
release(&kmem.lock);

/* kfree(): protected push */
acquire(&kmem.lock);
r->next = kmem.freelist;
kmem.freelist = r;
release(&kmem.lock);
```

Figure 3. Exact critical-section excerpts; surrounding validation and fills are omitted.

kfree() checks alignment and address range and fills the page before linking it. kalloc() fills the removed page after releasing the list lock. These details explain why the protected section is narrow. Locking does not make an invalid free correct; callers must still obey page ownership and lifetime rules.

10. Practical C workload and checking logic

The parent creates a pipe and forks four children. Each child waits for one byte from the pipe. After creation, the parent writes four tokens so workers can overlap. This start gate encourages concurrent requests; it does not guarantee simultaneous execution of any specific kernel instruction.

```
for (int round = 0; round < ROUNDS; round++) {
  char *allocation = sbrk(bytes);
  /* Check allocation failure. */
  volatile uchar *memory = (volatile uchar *)allocation;
  for (int offset = 0; offset < bytes; offset++)
    memory[offset] = tag;
  /* Brief user-space delay. */
  for (int offset = 0; offset < bytes; offset++)
    if (memory[offset] != tag)
      /* Report corruption and fail this child. */;
  /* Shrink by bytes; check failure. */
}
```

Figure 4. Explanatory workload excerpt; Appendix A contains the complete executable file.

Each child allocates eight pages, writes its own tag value from 1 to 4 into every byte, waits briefly in user space, verifies every byte, and releases the allocation. A changed byte generates a CORRUPTION message containing the round, offset, expected value, and actual value. The delay is not a kernel race-window modification.

The parent waits for child exit statuses. A successful result requires all four children to complete and no setup failure. Allocation, shrink, pipe, fork, and wait errors are handled. volatile forces the verification to reread memory in this program; it does not synchronize the allocator or repair its race.

A private user buffer is the observation point, while the shared kernel allocator is the modified component. Equal virtual addresses across processes are expected and do not prove that the same physical page was allocated twice.

11. Execution paths and reproduction

```
Positive sbrk()
  sys_sbrk -> growproc -> uvmalloc -> kalloc

Negative sbrk()
  growproc -> uvmdealloc / uvmunmap -> kfree
```

Figure 5. Allocation and release paths verified against the fixed source revision.

The public sbrk() wrapper uses eager allocation in this revision. Fork, page-table construction, and process exit also exercise the allocator. Therefore an unlocked failure may occur outside the worker verification loop. The syscall itself is not the lock; its kernel path reaches the protected resource.

Open the existing restored implementation

```
Windows PowerShell:
  wsl -d Ubuntu-24.04
Ubuntu shell:
  cd ~/xv6-member1
  make qemu CPUS=4
xv6 shell:
  race_test
Exit QEMU: Ctrl+A, then X
```

Reproduce a new measured run set

```
cc -std=c11 -Wall -Wextra -Werror -O2 \
  member1_tools.c -o /tmp/member1_tools
/tmp/member1_tools run /home/nithish/xv6-member1 \
  /home/nithish/member1-new-evidence full
```

The runner requires the installed race_test.c and its UPROGS entry, checks the original allocator, and refuses an existing evidence directory. Use a new output directory for a new run. These commands describe reproduction; they were not rerun during document preparation. Further runs must remain separate from the frozen results.

12. Possible allocator interleavings

The following schedules explain possible failures at source level. They are theoretical interleavings, not instruction traces recovered from the recorded experiments. The raw logs show symptoms but do not identify the exact internal schedule.

Table 8. Possible double allocation without locking

Step	CPU 0	CPU 1
1	Read head page P	
2		Read the same head P
3	Advance head to P.next	
4		Advance head to P.next
5	Return P	Return P

Both CPUs can observe the same head before either update excludes the other. In this simplified schedule, both return P. Correct locking makes each pop occur as a complete protected operation, so the second CPU observes the updated list instead.

Table 9. Possible lost free page without locking

Step	CPU 0 frees X	CPU 1 frees Y
1	Set X.next to old head H	
2		Set Y.next to old head H
3	Set head to X	
4		Set head to Y

The final chain is Y followed by H, and the new link to X has been overwritten. X is no longer reachable through the free-list head in this example. One lock around the complete push prevents the two insertions from overlapping in that way.

These examples describe why list invariants need coordination. They do not establish that a particular recorded corruption, allocation failure, or trap was caused by exactly one of these schedules. The test can miss lost pages or damage outside the buffers it verifies.

13. Original-lock baseline

The baseline establishes that the intended workload completes with the existing allocator protection under the chosen conditions. A separate sanity run reported ALL TESTS PASSED and a locked smoke check passed before measurement. The measured baseline then uses a fresh guest for each of A01-A10.

```
xv6 kernel is booting

hart 1 starting
hart 3 starting
hart 2 starting
init: starting sh
$ race_test
race_test: children=4 pages=8 rounds=200
race_test: PASS (all 4 children completed)
$ QEMU: Terminated
```

Figure 6. Rendered console excerpt from C-validation A01; this is saved text, not a desktop screenshot.

Table 10. Baseline checks

Check	Evidence / observation
Workload settings	children=4 pages=8 rounds=200
A01 result	PASS; all four children completed
A01-A10 phase total	10 PASS in 10 measured attempts
Control function	Confirms observed completion with original allocator locks

A baseline PASS means the program reported no detected error during that invocation. It does not establish absence of every possible defect. Its value is comparative: the next phase changes the allocator lock calls while holding the test and guest settings fixed.

The console displays hart startup messages because the guest uses four virtual CPUs. The order of those messages can vary. The recorded elapsed times include output collection and are not an allocator performance measurement.

14. Deliberate lock removal

The unlocked variant comments out exactly four allocator calls: acquire and release in kfree(), and acquire and release in kalloc(). The free-list update statements and the guest workload remain unchanged. The saved B-lock-removal.diff records the intended intervention.

```
kfree():
- acquire(&kmem.lock);
+ // acquire(&kmem.lock);
  r->next = kmem.freelist;
  kmem.freelist = r;
- release(&kmem.lock);
+ // release(&kmem.lock);

kalloc():
- acquire(&kmem.lock);
+ // acquire(&kmem.lock);
  r = kmem.freelist;
  if (r)
    kmem.freelist = r->next;
- release(&kmem.lock);
+ // release(&kmem.lock);
```

Figure 7. Focused excerpt of the saved allocator-lock removal diff.

After rebuilding, the runner performs B01-B10 with the same four-CPU stress program. The variant intentionally violates the normal locking discipline. This is a controlled teaching experiment rather than a proposed production optimization.

acquire() and release() also manage nested local interrupt state. Their removal changes that behavior wherever an outer lock is not already active. Consequently the comparison concerns the complete existing allocator locking discipline; it does not independently measure mutual exclusion, ordering, and interrupt handling as separate interventions.

Keep the removal diff and original allocator copy with the results so the experiment is auditable. Restoration follows this phase; the final project must retain its original protection.

15. Unlocked-run evidence

```
xv6 kernel is booting

hart 1 starting
hart 3 starting
hart 2 starting
init: starting sh
$ race_test
race_test: children=4 pages=8 rounds=200
child scause=0xd sepc=0x80000aca stval=0x505050505050505
panic: 2scause=0xd sepc=0x80000aca stval=0x505050505050505
panic: kerneltrapkerneltrap
: CORRUPTION round=1 offset=16384 expected=3 got=4
race_test: pid=6 failed status=1
QEMU: Terminated
```

Figure 8. Rendered C-validation B06 console excerpt; concurrent output is preserved as recorded.

B06 contains a visible CORRUPTION message: round=1, offset=16384, expected=3, got=4. The same log also contains interleaved trap and panic text. The supplied CSV assigns B06 the primary category CORRUPTION. The report retains this existing classification rather than counting that attempt again under panic/trap.

Table 11. Other recorded unlocked symptoms

Attempt	Recorded observation	Primary outcome
B01	panic: kerneltrap	PANIC_OR_TRAP
B07	race_test: FAIL (2 child failures; setup_failed=0)	TEST_FAILURE
B08	Host limit reached: 60 seconds	TIMEOUT
B10	race_test: FAIL (4 child failures; setup_failed=0)	TEST_FAILURE

Every attempt is counted once using its recorded primary outcome. Split or interleaved output should not be silently repaired into an invented complete message. A tag mismatch is direct evidence that the checked memory did not contain its expected value; it does not reveal the complete chain of internal kernel events.

The timeout records lack of completion before the host limit. It must not be relabeled as a proven deadlock. Similarly, panic/trap outcomes show severe failure but are not proof of one specific allocator interleaving.

16. Restoration and verification

After the unlocked phase, the exact saved original allocator is restored. The restoration check validates its checksum, the final allocator diff is empty, and the restored kernel is rebuilt. C01-C10 then repeat the same workload in fresh guests. This closes the experiment with the original synchronization in place.

```
restoration-check.txt:
kernel/kalloc.c: OK

final-restoration.diff:
(empty file)

test-verification.txt:
user/race_test.c: OK
```

Figure 9. Rendered verification excerpts from the C-validation evidence folder.

```
xv6 kernel is booting

hart 3 starting
hart 1 starting
hart 2 starting
init: starting sh
$ race_test
race_test: children=4 pages=8 rounds=200
race_test: PASS (all 4 children completed)
$ QEMU: Terminated
```

Figure 10. Rendered C-validation C01 console excerpt after allocator restoration.

All ten restored attempts reported PASS. The baseline and restored phases therefore show the same observed completion count, while the unlocked phase shows no PASS outcomes under this workload. The return to passing results supports the importance of the original locking discipline in this controlled comparison.

The checksum and empty diff are evidence of restoration at the time the run set was collected. They are not a claim that this report preparation performed a new kernel build. The preserved final-restored-build.txt contains the recorded final build output.

17. Results and interpretation

Table 12. C-validation outcomes; each attempt counted once

Phase	Runs	PASS	Corruption	Panic / trap	Test failure	Timeout
Original locks	10	10	0	0	0	0
Locks removed	10	0	1	6	2	1
Locks restored	10	10	0	0	0	0

The normal and restored allocator phases each produced ten PASS outcomes. All ten unlocked attempts produced a non-PASS primary outcome. The comparison supports retaining the original locking discipline under the tested configuration. It does not establish a universal numerical failure probability for other workloads or computers.

How the evidence supports the explanation

Source analysis identifies free-list updates that require a coherent relationship between the head and links. The controlled diff removes their common protection. The observed failures appear in that phase, and observed completion returns after restoration. This provides a useful practical connection between the invariant, its synchronization mechanism, and the recorded behavior.

Limitations

The test verifies its own byte buffers and process completion; it can miss lost pages, overwritten corruption, or damage elsewhere. The logs are not an instruction-level trace. Failure messages can interleave, and primary categories summarize attempts with more than one symptom. Removing the lock calls also removes their local interrupt-management effects in some contexts.

The data covers one revision, one workload, four virtual CPUs, and ten attempts per phase. Elapsed time includes host-side output collection, so it should not be used as a speed comparison. An unlocked pass would not prove safety, and the absence of locked failures is evidence limited to the tested attempts.

Demonstration material

The accompanying PPT links to demo-walkthrough.mp4. That file is an earlier-run walkthrough rendered from saved source and logs, not a live screen recording and not a recording of this C-validation set. Use it to explain the procedure; use Tables 12-14 and the C-validation logs for the report results.

18. Complete attempt record: A01-B05

The following entries are transcribed from the C-runner results.csv. Each attempt used four CPUs, four children, eight pages per child, and 200 rounds. All attempts reached the shell and started the workload. Seconds are recorded host elapsed values, not benchmark results.

Table 13. First 15 measured attempts

Attempt	Phase	Primary outcome	Seconds
A01	locked	PASS	0.857
A02	locked	PASS	0.848
A03	locked	PASS	0.847
A04	locked	PASS	0.864
A05	locked	PASS	0.843
A06	locked	PASS	0.862
A07	locked	PASS	0.841
A08	locked	PASS	0.846
A09	locked	PASS	0.855
A10	locked	PASS	0.852
B01	unlocked	PANIC_OR_TRAP	0.780
B02	unlocked	PANIC_OR_TRAP	0.779
B03	unlocked	PANIC_OR_TRAP	0.780
B04	unlocked	PANIC_OR_TRAP	0.778
B05	unlocked	PANIC_OR_TRAP	0.777

The raw console log for an attempt is named with its ID and phase, for example A01-locked.txt or B01-unlocked.txt, inside member1_completed/c_validation. The companion .cast files preserve timed terminal output. The CSV stores the exact selected message and evidence filenames for every entry.

B03-B05 contain split or interleaved diagnostics. Their existing primary categories remain PANIC_OR_TRAP. The original earlier-run CSV is not included in this table, even though it uses the same attempt-ID pattern.

19. Complete attempt record: B06-C10

Table 14. Remaining 15 measured attempts

Attempt	Phase	Primary outcome	Seconds
B06	unlocked	CORRUPTION	0.792
B07	unlocked	TEST_FAILURE	0.852
B08	unlocked	TIMEOUT	60.028
B09	unlocked	PANIC_OR_TRAP	0.762
B10	unlocked	TEST_FAILURE	0.865
C01	restored	PASS	0.876
C02	restored	PASS	0.852
C03	restored	PASS	0.864
C04	restored	PASS	0.886
C05	restored	PASS	0.872
C06	restored	PASS	0.868
C07	restored	PASS	0.876
C08	restored	PASS	0.889
C09	restored	PASS	0.867
C10	restored	PASS	0.859

B06 is counted as CORRUPTION according to the supplied CSV despite additional panic/trap text in its raw log. B08 is counted as TIMEOUT because the host limit was reached. Each row is one attempt, not one symptom. C01-C10 are measured after restoration of the original allocator.

Reading and preserving the evidence

Preserve results.csv together with its matching raw logs, recordings, source copies, protocol, and restoration checks. The local SHA-256 manifests support integrity checking. If further experiments are run, save them to a new folder and report their settings and results separately.

A presentation can use the phase summary for readability, while these tables keep every attempt available for review. The accompanying chart and table use these same counts. No successful or unsuccessful measured attempt has been omitted from the report.

20. Conclusion, applications and future scope

This case study connects race-condition theory with a concrete kernel data structure. Four concurrent C workers repeatedly allocate and release memory through xv6. Standard xv6 protects the shared physical-page free list with kmem.lock. Temporarily removing its existing calls exposes unsafe concurrent interaction; restoring them returns the tested workload to observed completion.

In the C-validation set, original locks passed in 10 of 10 attempts, the unlocked variant passed in 0 of 10, and restored locks passed in 10 of 10. These results support the original locking discipline for the tested conditions. They do not identify an exact failing instruction schedule or prove the absence of all races.

What was learned

A short source operation may contain several machine-level actions. Synchronization must protect the invariant across the whole critical section. Atomic ownership operations and memory ordering support the lock, while nested interrupt handling addresses local execution constraints. All relevant accesses must obey the same rule; a lock around only one caller is insufficient.

Different synchronization types serve different purposes. Mutexes and spinlocks control ownership, condition variables coordinate predicates, semaphores manage permits, and barriers align phases. Using a primitive with the wrong protocol can leave a race or introduce deadlock. The comparison methods were explained conceptually; the experimental mechanism was the allocator spinlock.

Applications

The same reasoning applies to operating-system memory lists, shared queues, caches, file metadata, and application state such as inventory or account updates. In each case, identify the invariant, choose a coordination rule suitable for the context, and check that all participants follow it. Particular applications need their own implementation and validation.

Future scope

Further work could vary CPU count and workload size, add checks for lost pages, investigate another shared kernel structure, or measure lock contention with an appropriate benchmark. Per-CPU allocator designs could reduce contention but require careful transfer rules. These extensions have not been performed in this case study.

References

[1] Local experimental evidence, member1_completed/c_validation: results.csv, A01-C10 console logs and recordings, protocol.txt, B-lock-removal.diff, source copies, test-verification.txt, restoration-check.txt, and final-restoration.diff. Collected 24 October 2026.

[2] MIT PDOS, xv6-riscv, fixed revision f780d7850765d74cbdcd1fff4b822d8829f7c9bd. Examined files: kernel/kalloc.c, kernel/spinlock.c, kernel/sleeplock.c, kernel/proc.c, kernel/sysproc.c, kernel/vm.c, and user/ulib.c. https://github.com/mit-pdos/xv6-riscv/tree/f780d7850765d74cbdcd1fff4b822d8829f7c9bd

[3] MIT PDOS, xv6: a simple, Unix-like teaching operating system, Locking chapter. Background reference; local fixed source controls exact implementation details. https://mit-pdos.github.io/xv6-riscv-book/lock.html

[4] POSIX documentation reproduced by the Linux man-pages project: pthread_cond_wait(3p), sem_wait(3), pthread_rwlock_rdlock(3p), and pthread_barrier_wait(3p). https://man7.org/linux/man-pages/

[5] race_test.c and member1_tools.c, supplied case-study workload and host helper. Full guest workload is reproduced in Appendix A; the host helper is retained with the project.

[6] Faculty-provided handwritten case-study format, 1000097330.pdf, two pages. Specifies report typography and presentation structure.

[7] Earlier-run demonstration: member1_completed/demo-walkthrough.mp4. A rendered source/log walkthrough; separate from C-validation results and not a live desktop recording.

Web documentation consulted on 25 October 2026. Report evidence excerpts are rendered from saved text. No generated image is presented as a native screenshot.

Appendix A. Complete C workload (1/3)

Executed guest source: race_test.used.c. This is the supplied C-validation copy; the code is reproduced without functional changes. Lines continue on the following pages.

```
// User-space stress test for the xv6-riscv physical-page allocator.
// Add $U/_race_test\ to UPROGS in the xv6 Makefile.
// This test detects some corruption; a PASS is not a proof of safety.
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

#define NCHILD 4
#define NPAGES 8
#define ROUNDS 200
#define PAGE_BYTES 4096

static int
worker(int id)
{
  const int bytes = NPAGES * PAGE_BYTES;
  const uchar tag = (uchar)(id + 1);

  for (int round = 0; round < ROUNDS; round++) {
    char *allocation = sbrk(bytes);
    if (allocation == (char *)-1) {
      printf("child %d: allocation failed at round %d\n", id, round);
      return 1;
    }

    // Volatile forces the verification to read memory again.
    // It does not synchronize the kernel or fix the allocator race.
    volatile uchar *memory = (volatile uchar *)allocation;
    for (int offset = 0; offset < bytes; offset++)
      memory[offset] = tag;

    // User-space delay gives other workers an opportunity to overwrite
    // an incorrectly shared page. It is not a kernel race-window delay.
    for (volatile int delay = 0; delay < 20000; delay++)
      ;

    for (int offset = 0; offset < bytes; offset++) {
      int actual = memory[offset];
      if (actual != tag) {
        printf("child %d: CORRUPTION round=%d offset=%d expected=%d got=%d\n",
               id, round, offset, (int)tag, actual);
        // exit() will reclaim this child's remaining address space.
        return 1;
      }
```

Appendix A. Complete C workload (2/3)

```
    }

    if (sbrk(-bytes) == (char *)-1) {
      printf("child %d: shrink failed at round %d\n", id, round);
      return 1;
    }
  }
  return 0;
}

int
main(void)
{
  int gate[2];
  int started = 0;
  int setup_failed = 0;
  int bad = 0;

  printf("race_test: children=%d pages=%d rounds=%d\n",
         NCHILD, NPAGES, ROUNDS);

  if (pipe(gate) < 0) {
    printf("race_test: pipe setup failed\n");
    exit(1);
  }

  for (int id = 0; id < NCHILD; id++) {
    int pid = fork();
    if (pid < 0) {
      printf("race_test: fork failed after %d children\n", started);
      setup_failed = 1;
      break;
    }
    if (pid == 0) {
      char token;
      close(gate[1]);
      // Each child waits for one byte before starting its work.
      int received = read(gate[0], &token, 1);
      close(gate[0]);
      if (received != 1)
        exit(2);
      exit(worker(id));
    }
    started++;
```

Appendix A. Complete C workload (3/3)

```
  }

  close(gate[0]);
  if (!setup_failed) {
    char tokens[NCHILD];
    for (int i = 0; i < NCHILD; i++)
      tokens[i] = 'G';
    if (write(gate[1], tokens, NCHILD) != NCHILD) {
      printf("race_test: start gate failed\n");
      setup_failed = 1;
    }
  }
  // On setup failure, EOF lets already-created children exit.
  close(gate[1]);

  for (int i = 0; i < started; i++) {
    int status = -1;
    int pid = wait(&status);
    if (pid < 0) {
      printf("race_test: wait failed\n");
      setup_failed = 1;
      break;
    }
    if (status != 0) {
      printf("race_test: pid=%d failed status=%d\n", pid, status);
      bad++;
    }
  }

  if (setup_failed || bad || started != NCHILD) {
    printf("race_test: FAIL (%d child failures; setup_failed=%d)\n",
           bad, setup_failed);
    exit(1);
  }

  printf("race_test: PASS (all %d children completed)\n", NCHILD);
  exit(0);
}
```